

HiDiTingV100 AEF overlay

开发指南

文档版本	00B01
发布日期	2025-06-05

前言

概述

本文档主要介绍 AEF overlay 集成调试相关内容，包括使用方法和注意事项，用于指导客户更换使用第三方 AEF 算法。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 概述.....1

2 算法集成开发包.....2

2.1 集成开发包说明..... 2

2.1.1 build 目录 2

2.1.2 include 目录 2

2.1.3 overlay 目录 3

2.1.4 component 目录..... 3

2.2 开发环境配置 3

2.3 编译命令..... 4

2.4 增加或删除 C 库函数..... 4

2.5 示例插件介绍 4

2.5.1 插件目录结构 5

2.5.2 插件编译脚本配置 5

3 创建 AEF 插件工程.....7

3.1 AEF 插件工程命名要求..... 7

3.2 新建 AEF 插件工程目录 7

3.3 AEF 插件编译脚本 8

4 开发 AEF 插件.....9

4.1 AEF 插件接口说明 9

4.2 AEF 插件适配样例 10

4.3 AEF 插件注册方法11

5 加载和运行算法.....12

5.1 生成算法镜像包..... 12

5.2 启动 Sample_a0 程序加载和运行算法 12

Draft

插图目录

图 5-1 烧写完成示意图..... 13

表格目录

表 4-1 音效算法 audio_frame 结构体说明 10

1 概述

本文档主要介绍 AEF overlay 集成相关内容，重点介绍了算法集成开发包、开发算法环境配置、开发算法插件、加载和运行算法等。AEF (AUDIO EFFECT) 算法是指音效算法或音频后处理算法。

2 算法集成开发包

2.1 集成开发包说明

2.2 开发环境配置

2.3 编译命令

2.4 增加或删除 C 库函数

2.5 示例插件介绍

2.1 集成开发包说明

集成开发包所在路径：工程根目录/application/brandy/audio/sample_dsp_overlay/aeF

2.1.1 build 目录

build 目录：

- xtensa 子目录：存放 DSP 相关的配置文件和编译脚本，建议只引用不修改。
 - hifi_rules.mak：配置编译规则、overlay 打包规则和依赖关系。
 - hifi_lsp.mak：配置 overlay 打包规则，被 hifi_rules.mak 调用。
 - xtensa_cfg.mak：配置 HiFi 工具链和 overlay 打包工具。
- xtensa/arch 子目录：各个 overlay 的 memmap 配置。
- xtensa/tool 子目录：overlay 打包工具。

2.1.2 include 目录

include 目录：

td_type.h: C 语言变量类型重定义。

2.1.3 overlay 目录

overlay 目录:

- component 头文件, 必须在插件源文件中包含。
component.h: 通用算法插件注册接口, 通过 define_component 接口将插件 entry 地址注册到.component 内存段中。
- core_overlay 模块作为插件统一入口和 C 库依赖, 被编译到各个算法插件的 bin 文件中。
core_overlay.c: 重定向 C 库函数到相应的钩子函数, 以及将 overlay 引导地址.start 指向可访问.component 内存的接口。

2.1.4 component 目录

component 目录:

- aef/plugins 子目录: AEF 插件工程目录。
 - aef_lib0: AEF 插件示例工程, 开发者可参考该工程创建新的 AEF 插件工程。

2.2 开发环境配置

进入集成开发包所在路径: 工程根目录
/application/brandy/audio/sample_dsp_overlay/aef

- 工程路径配置

sap_idp.mak 路径: 工程根目录

/application/brandy/audio/sample_dsp_overlay/aef/sap_idp.mak。

修改 sap_idp.mak:

- IDP_OUT_DIR: 编译输出目录, 可修改到开发者自定义目录, 建议使用默认目录。

- Xtensa 编译工具链配置

xtensa_cfg.mak 路径: 工程根目录

/application/brandy/audio/sample_dsp_overlay/aef/build/xtensa/xtensa_cfg.mak 。

修改 xtensa_cfg.mak:

- XTENSA_HOME：配置 HiFi 编译工具链安装路径。
- XTENSAD_LICENSE_FILE：配置 HiFi 工具链 XTENSA License 文件。

2.3 编译命令

在算法集成开发包所在路径下，执行以下命令：

步骤 1 清除历史编译结果：make clean

步骤 2 编译算法静态库：make lib

步骤 3 编译算法插件 bin 文件：make all

----结束

2.4 增加或删除 C 库函数

说明

如果主镜像未注册相应的 C 库函数到该新增的“overlay_func->func”函数指针，会导致 overlay 新增 C 库函数钩子。开发者可自行实现 C 库函数，或是知会主镜像发布者发布新的镜像将相应的 C 库函数注册到“overlay_func->func”函数指针。

修改集成开发包的 overlay 目录下的“core_overlay.h”文件和“core_overlay.c”文件：

```
/* 在core_overlay.h的overlay_func结构体中，新增一个C库同名函数类型定义 */
typedef struct {
    int (*func)(void);
} overlay_func;
/* 在core_overlay.c源码中，新增一个C库同名函数实现，重定向到主镜像的钩子上 */
int func(void)
{
    return g_overlay_func->func();
}
```

2.5 示例插件介绍

示例插件路径：工程根目录

/application/brandy/audio/sample_dsp_overlay/aef/component/aef/plugins/aef_lib0

2.5.1 插件目录结构

component/aef/plugins/aef_lib0 工程目录结构如下：

- aef_lib0/include 目录：放置 AEF 算法的头文件。
- aef_lib0/lib 目录：放置 AEF 算法的依赖库文件。
- aef_lib0/src 目录：放置 AEF 算法的源码文件。
- aef_lib0/xaef_lib0_comp.c 源文件：AEF 算法适配样例。
- aef_lib0/xaef_lib0_comp.h 头文件：AEF 算法适配头文件。

本示例插件仅包含“xaef_lib0_comp.c”源文件和“xaef_lib0_comp.h”头文件。
include 目录、lib 目录、src 目录仅作为目录结构参考，开发者创建算法工程时，可按如上目录结构说明放置文件。

2.5.2 插件编译脚本配置

- 在 component/aef/plugins/Makefile 编译脚本中，增加新增的 AEF 插件 aef_lib0：

```
=====
===
aef_libx = aef_lib0
=====
===
```

- component/aef/plugins/aef_lib0/Makefile 编译脚本内容如下：

```
CUR_DIR := $(shell if [ "$$PWD" != "" ]; then echo $$PWD; else pwd; fi)
# 从当前目录回退到集成开发包所在路径
IDP_TOP_DIR := $(realpath $(CUR_DIR)/../../..)
# 包含集成开发包编译环境配置和编译工具链配置文件
include $(IDP_TOP_DIR)/sap_idp.mak
include $(IDP_BUILD_DIR)/xtensa_cfg.mak
# 编译待集成的 AEF 算法源码成静态库 libaef_lib0.a
ifdef BUILD_LIB
TARGET := libaef_lib0
CFLAGS_EXT :=
INCLUDES := -linclude -I$(IDP_TOP_DIR)/include
LIB_INSTALL_DIR := lib
/* 只编译待集成 AEF 算法的源文件 */
SRCFILE := $(wildcard src/*.c)
# 编译待集成的 AEF 插件，并链接算法静态库 libaef_lib0.a 到 aef_lib0.bin 中
```

```
else
TARGET := aef_lib0
CFLAGS_EXT :=
INCLUDES := -linclude -I$(IDP_TOP_DIR)/include
# 链接待集成AEF算法的静态库 libaef_lib0.a, 如还依赖其他静态库也链接进来。
LIBS := -l:lib/libaef_lib0.a
# 只编译待集成AEF算法的插件适配源文件
SRCFILE := xaaef_lib0_comp.c
endif
# 包含编译规则和打包规则配置文件
include ../rules.mak
```

3

创建 AEF 插件工程

3.1 AEF 插件工程命名要求

3.2 新建 AEF 插件工程目录

3.3 AEF 插件编译脚本

3.1 AEF 插件工程命名要求

AEF 插件镜像 bin 文件命名要求：AEF 插件工程命名建议与插件镜像 bin 文件名一致。如工程命名为“aef_lib0”，则插件镜像命名为“aef_lib0.bin”。

3.2 新建 AEF 插件工程目录

请参考“[2.5.1 插件目录结构](#)”，新建工程 aef_lib1。

新建工程 aef_lib1 到 component/aef/plugins，工程目录结构如下：

- aef_lib1/include 目录：放置 AEF 算法的头文件。
- aef_lib1/lib 目录：放置 AEF 算法的依赖库文件。
- aef_lib1/src 目录：放置 AEF 算法的源码文件。
- aef_lib1/xaef_lib1_comp.c 源文件：AEF 算法适配源文件。
- aef_lib1/xaef_lib1_comp.h 头文件：AEF 算法适配头文件。

3.3 AEF 插件编译脚本

在 component/aef/plugins/Makefile 编译脚本中，增加新增的 AEF 插件命令如下：

```
aef_libx += aef_lib1
```

请参考“[2.5.2 插件编译脚本配置](#)”示例插件

component/aef/plugins/aef_lib0/Makefile 的编译脚本，创建新增插件 aef_lib1 的编译脚本。

4 开发 AEF 插件

4.1 AEF 插件接口说明

4.2 AEF 插件适配样例

4.3 AEF 插件注册方法

4.1 AEF 插件接口说明

AEF 插件接口说明如下：

- xaaef_lib0_comp.h 定义结构体 aef_component 成员变量（即 AEF 插件属性）：
 - reserved：保留，不用配置。
 - name：字符串，算法库名称。
 - aef_type：AEF 插件类型，参考枚举类型 aef_type。
 - version：版本。
- xaaef_lib0_comp.h 定义结构体 aef_component 成员接口（即 AEF 插件适配接口）：
 - create：创建算法实例并输出唯一的实例句柄给 AEF。（必备接口）
 - destroy：销毁实例句柄对应的算法实例，并释放相关资源。（必备接口）
 - set_config：设置算法配置。（可选接口）
 - get_config：获取算法配置。（可选接口）
 - set_parameter：设置实例句柄对应的算法实例的参数。（可选接口）
 - get_parameter：获取实例句柄对应的算法实例的参数。（可选接口）
 - set_enable：使能或停止实例句柄对应的算法实例。（可选接口）

- get_enable：获取实例句柄对应的算法实例的使能状态。（可选接口）
- get_max_pcm_in_size：获取输入 PCM 数据的最大帧长。（可选接口）
- get_max_pcm_out_size：获取输出 PCM 数据的最大帧长。（可选接口）
- get_input_pcm_attr：获取输入 PCM 数据的属性。（必备接口）
- proc_frame：执行实例句柄对应的算法实例来处理输入的 PCM 数据。（必备接口）

表4-1 音效算法 audio_frame 结构体说明

属性项	说明
interleaved	数据是否交织。
channels	声道。
bit_depth	位宽。
sample_rate	采样率。
pts	帧时间戳。
pcm_buffer	未使用，待扩展。
bits_buffer	音频 PCM 数据缓存指针。
bits_bytes	音频 PCM 数据大小。
pcm_samples	音频 PCM 数据采样点数。
frame_index	音频帧索引编号。
eos	帧结束标志。
pkg_loss	丢包标志。

4.2 AEF 插件适配样例

请参考算法集成开发包路径下的适配样例：

```
工程根目录
/application/brandy/audio/sample_dsp_overlay/aeff/component/aeff/plugins/aeff_lib0/xa
aeff_lib0_comp.c
```

4.3 AEF 插件注册方法

AEF 插件注册参考样例及说明如下。

```
/* 定义一个全局的插件 entry结构体 */
static aef_component ha_aef_lib_entry = {
    .name = (const td_char *)"aef_lib0",
    .type = AEF_TYPE_XXX,
    .create = aef_lib_create,
    .destroy = aef_lib_destroy,
    .set_config = aef_lib_set_config,
    .get_config = aef_lib_get_config,
    .set_parameter = aef_lib_set_parameter,
    .get_parameter = aef_lib_get_parameter,
    .set_enable = aef_lib_set_enable,
    .get_enable = aef_lib_get_enable,
    .get_max_pcm_in_size = aef_lib_get_max_pcm_in_size,
    .set_max_pcm_in_size = aef_lib_set_max_pcm_in_size,
    .get_input_pcm_attr = aef_lib_get_input_pcm_attr,
    .proc_frame = aef_lib_proc_frame,
};
/* 将插件entry地址注册到.component地址段中 */
define_component(AEF, ha_aef_lib_entry)
```

说明

自定义的 entry 结构体中，成员变量 type 必须定义为 AEF_TYPE_SMARTPA 或者 AEF_TYPE_SWS1。

5 加载和运行算法

5.1 生成算法镜像包

5.2 启动 Sample_ao 程序加载和运行算法

5.1 生成算法镜像包

例如，在 DSP 版本类型为 max、AEF 插件镜像 bin 文件为“aef_lib0.bin”的情况下：
在算法集成开发包所在路径中执行命令“./aef_replace_overlay.sh max aef_lib0.bin”。

打包生成的“dsp_overlay.bin”文件的路径：工程根目录
/tools/pkg/bin/brandy/dsp/max

说明

命令参数说明：./aef_replace_overlay.sh param1 param2

- param1：DSP 的版本类型。如：max、mini（当前 target1 对应 mini，target3/4/5 对应 max）。
- param2：生成的 AEF 插件镜像 bin 文件名称，如：aef_lib0.bin。

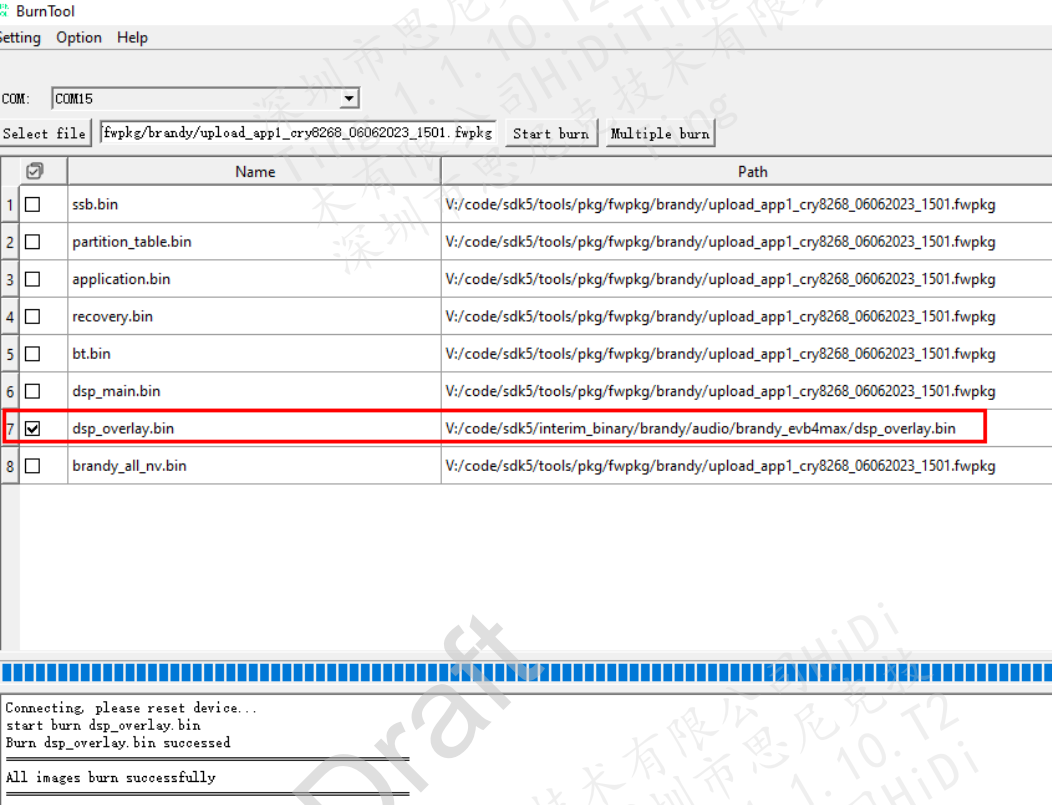
5.2 启动 Sample_ao 程序加载和运行算法

步骤 1 烧录“dsp_overlay.bin”文件到指定的 Flash 分区：

1. 启动 BurnTool 工具，完成一次整机镜像烧写。具体烧写步骤请参见《HiDiTingV100 BurnTool 工具 使用指南》。
2. 如图 5-1 所示，点击“dsp_overlay.bin”对应的 Path 文本框，选择“5.1 生成算法镜像包”中打包生成的“dsp_overlay.bin”文件的路径。

3. 点击 “Start burn” 启动烧写 “dsp_overlay.bin” 。

图5-1 烧写完成示意图



步骤 2 执行 “sample_ao” 命令测试插件

输入 AT 命令：AT^audio=sample_ao /user/stream/pcm/PCM_16k_1ch_16bit.pcm 0 16000 16 1

(命令中的 “/user/stream/pcm/PCM_16k_1ch_16bit.pcm” 需提前上传至文件系统，或替换为文件系统中已有的 pcm 文件)

步骤 3 测试结果：能够正常播放并且声音无异常。

----结束